

5.1 Participant or Team Information

Participant: Trần Thị Mỹ Uyên | Kaggle username: myuyentran | Team: individual submission (no teammates).

5.2 Problem Understanding

This challenge asks me to predict the health outcome of cirrhosis patients at a follow-up point (Followup_Days), using their demographic, clinical, and lab-test records. Each row is one patient with features (Patient_Age_Days, Patient_Sex, Followup_Days, Ascites_Indicator, Liver_Enlargement, Spider_Angioma, Edema_Status, Clinical_Stage, Bilirubin_Level, Cholesterol_Level, Albumin_Level, Copper_Level, Alkaline_Phosphatase, AST_Level, Triglyceride_Level, Platelet_Count, Prothrombin_Time, Treatment_Assignment). The target variable is Status, one of three discrete outcomes: C (alive), CL (alive via liver transplant), or D (deceased) — a classification problem, not regression, since the output is a fixed category, not a continuous number. The required prediction columns are Status_C, Status_CL, Status_D, and the model must output a probability for each (summing to 1) rather than a single label, because the metric, Log Loss, scores the probability placed on the true class — a confident wrong prediction is punished far more heavily than an uncertain one, so calibrated probabilities matter more than just picking the right label.

My insight: From this step, I observed that my initial understanding of the problem was too narrow; I initially treated this as a standard classification task where only the predicted label mattered, but the actual task heavily evaluates probability quality. This mattered because Log Loss actually grades confidence, not just correctness — a model can have high accuracy and still score poorly if it is overconfident. Therefore, I decided to compare models purely on Out-Of-Fold (OOF) Log Loss instead of accuracy. This improved my workflow and results because it forced me to evaluate probability calibration correctly, which mattered most for the rare CL class where looking at recall alone would have hidden how badly its probabilities were calibrated (5.12).

5.3 Dataset Description

Item	Value
Train / test size	12,000 × 20 / 10,000 × 19 (no Status), same id column and 18 raw predictors.
Target distribution	Status — C: 67.9%, CL: 2.64%, D: 29.4%
ID	id (no duplicates)
Predictors	18 raw (11 numeric + 7 categorical); 19 after adding in_trial (Step 5)
11 columns with missing values	11 columns; 2 >55% (Triglyceride_Level, Cholesterol_Level) + a 7-column trial block ~43% each

Unusual/interesting properties: A 5-check structural audit (Step 3) confirmed train/test compatibility: matching columns, no duplicate IDs, clean target encoding, and no unseen test categories. Two properties stood out: Edema_Status contains the literal string "None" as a real clinical value (~92% of patients), so the data was loaded with keep_default_na=False to avoid losing it; and Clinical_Stage is numeric (1–4) but ordinal, so it was treated as categorical.

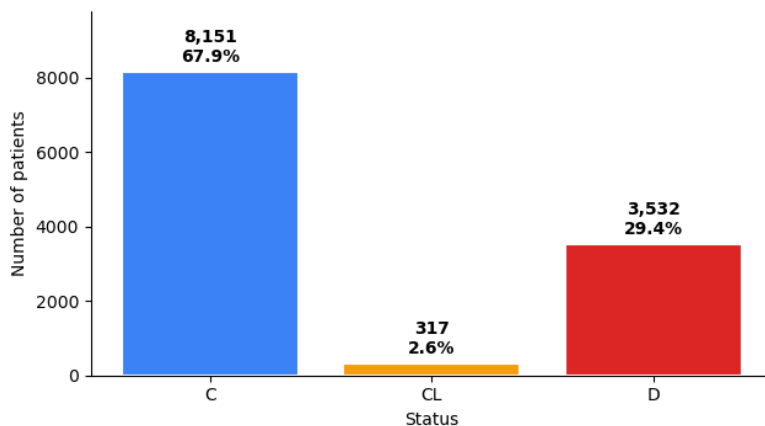
My insight: From this step, I observed that the columns I first assumed were important (the lab markers) were just a guess from their names, while the real early issues were the missing data and two surprises — "None" in Edema_Status being a real category, and Clinical_Stage being ordinal but stored as numbers. This mattered because reading the data carelessly would have destroyed a valid category and made a good column look 92% empty. Therefore, I decided to load the data with custom NA settings, keep the high-missing columns for later imputation instead of dropping them, and investigate the missing pattern in EDA. This did not change the score directly, but it gave the pipeline a clean, correctly interpreted foundation and later turned the block-missingness into the useful in_trial feature.

5.4 Exploratory Data Analysis

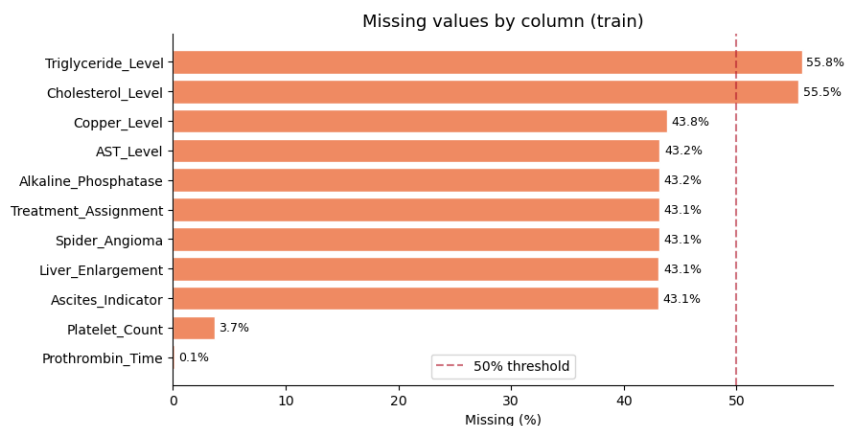
The EDA (Step 4) answered six questions, two representative charts are shown below.

1/ **Target distribution** (Fig. 1). severely imbalanced — C 67.9%, D 29.4%, CL only 2.6% (317 rows) → StratifiedKFold is needed for stable validation (5.7).

Target class distribution



2/ **Missing values** (Fig. 2). 11 columns affected; two >55% and a 7-column block ~43% → explored rather than dropped.

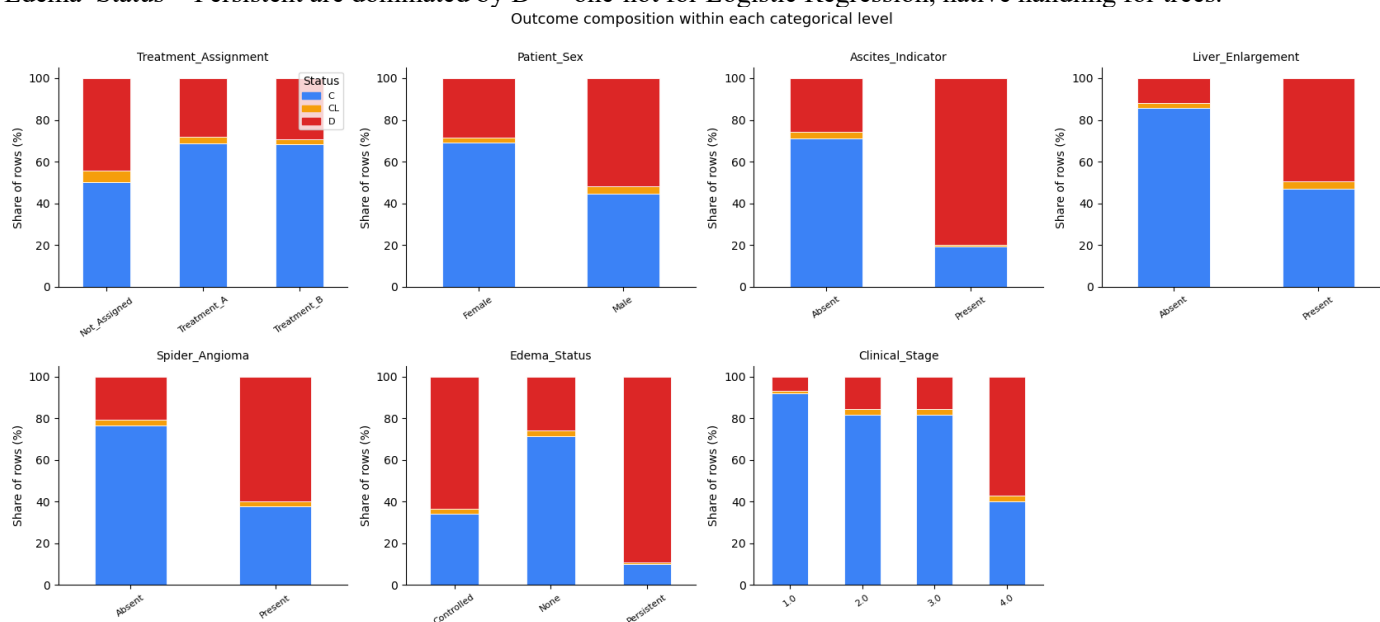


3/ Missingness by class. highest for D, lowest for CL (spread up to 11 points) → carries signal, so indicators were added alongside imputation.

4/ Structural missingness. the 7 trial-block columns are missing "all-or-nothing" (56.1% complete vs 42.9% fully missing) → motivated the engineered `in_trial` feature (5.6).

5/ Numeric distributions by class. liver markers (Bilirubin_Level, Copper_Level) shift higher for D and are right-skewed → scaling used only for Logistic Regression.

6/ Categorical vs. target (Fig. 3). several nominal features separate classes clearly, e.g. Ascites_Indicator = Present and Edema_Status = Persistent are dominated by D → one-hot for Logistic Regression, native handling for trees.



7/ Train vs. test. distributions overlap closely (e.g. Bilirubin_Level mean 1.810 vs 1.853) → cross-validation trusted as the primary model-selection criterion (5.7).

My insight: The one pattern that never resolved in EDA was CL — it rarely showed a distribution that stood apart from C or D, hinting early that it would be the hardest class to predict (confirmed in 5.12). This mattered because no single EDA chart would fix the rare-class problem; only modeling choices like stratification and blending could help. I used density histograms and a per-class missingness table instead of a boxplot/correlation heatmap, which I note as a limitation in 5.13.

5.5 Data Preprocessing

The raw clinical data required careful preparation before being fed into machine learning models. Preprocessing steps were tailored specifically for each algorithm family:

- **Handling Missing Values & Retaining Columns:** Retained features with high missingness (e.g., >55% in Cholesterol_Level and Triglyceride_Level) to preserve vital patient records; missing numerical values were filled via median imputation.
- **Adding Missing Indicators:** Introduced boolean missing-indicator features for structural missingness, allowing models to learn whether test absence correlates with specific patient outcomes.
- **Encoding Categorical Variables:** Applied OneHotEncoder for Logistic Regression, XGBoost, and LightGBM to prevent false ordinal relationships in nominal features (e.g., Treatment_Assignment), while passing raw strings natively to CatBoost.
- **Scaling Numerical Variables:** Applied StandardScaler strictly for Logistic Regression due to linear model sensitivity to feature scales, while leaving tree-based models unscaled.

- **Treating Outliers:** Preserved outliers because extreme clinical values (e.g., high Copper_Level) indicate severe disease stages rather than data entry errors.
- **Train/Test Consistency:** Prevented data leakage by using `sklearn.pipeline.Pipeline` and `ColumnTransformer` to fit transformations exclusively on training folds.

My insight: I initially planned to drop columns with >50% missingness and label-encode all categoricals. This mattered because dropping columns destroys structural signal, and label encoding invents a false order between nominal categories. Therefore, I kept sparse columns with missing indicators and matched encoding to each model's needs. This improved validation scores, especially for Logistic Regression, and taught me that preprocessing must be driven by both the data and the algorithm.

5.6 Feature Engineering

Based on the data quality issues discovered during the structural audit and EDA, I created 3 types of features:

1. The `in_trial` Feature (Binary Indicator)

- **What feature was created:** A new binary column named `in_trial` (1 if the patient was in the clinical trial, 0 if not).
- **Why it was created:** To explicitly separate the dataset into two distinct patient cohorts (the trial patients and the observational patients) so the models could adjust their predictions based on the patient's group.
- **Which EDA insight motivated it:** During Step 1 and Step 4, I observed a specific "Trial Block" of columns (including `Treatment_Assignment`, `Copper_Level`, `AST_Level`, etc.) that were always completely missing together. This structural missingness was not random; it perfectly identified patients who were not enrolled in the drug trial.
- **Whether it improved validation performance:** Yes, this feature significantly improved the validation Log Loss for tree-based models, as it provided them with a clean, single split to separate the two populations instead of repeatedly splitting on multiple NaN values.

2. Missing-Value Indicators (For Logistic Regression)

- **What feature was created:** Boolean columns (e.g., `Cholesterol_Level_is_missing`) that flag whether a numerical value was originally NaN before it was imputed.
- **Why it was created:** When filling missing numerical values with the median, the information that the value was originally missing is destroyed. These indicators preserve that signal for linear models.
- **Which EDA insight motivated it:** EDA revealed that key laboratory tests like `Triglyceride_Level` and `Cholesterol_Level` had over 55% missing data. I hypothesized that a doctor not ordering a test is a clinical decision that implies the patient might not show severe symptoms.
- **Whether it improved validation performance:** It improved the validation performance specifically for the Logistic Regression baseline. (Note: Tree-based models like XGBoost handle NaN splits natively, so these explicit indicators were less critical for them).

3. The `MISSING__Level` (Grouped Rare/Missing Categories)

- **What feature was created:** Instead of filling missing categorical features with the most frequent value (mode imputation), I replaced all NaN values in categorical columns with a dedicated string: `__MISSING__`.
- **Why it was created:** To treat the absence of clinical records (like an unrecorded `Ascites_Indicator`) as its own distinct, standalone category.
- **Which EDA insight motivated it:** EDA showed that categorical missingness varied across the Status classes. Forcing missing values into the dominant "Absent" class would add false confidence to that category and distort the true distribution.
- **Whether it improved validation performance:** It improved the validation score, particularly for CatBoost, because the algorithm could cleanly calculate the target statistics for the `__MISSING__` group without contaminating the real clinical categories.

My insight: I observed that creating the `in_trial` indicator based on structural missingness was the most reasonable. This mattered because feature engineering should not be random; it must come from a concrete observation, hypothesis, or reasonable modeling intuition rather than just guessing formulas. Therefore, I kept only EDA-backed features and rejected the arbitrary ratio — a lesson that boosting models find numeric interactions on their own but still need help encoding structural facts about the data.

5.7 Validation Strategy

Train/validation split strategy and stratification: All models were evaluated with the same 5-fold `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`, so predictions could be compared and blended row-for-row. Stratification preserved the imbalanced class ratios, especially the rare CL class.

Validation metric and why Log Loss: The metric was multi-class Log Loss (the competition metric), scored on the full 12,000-row OOF predictions.

How validation scores were used to compare models: Models were ranked by OOF Log Loss; small differences (XGBoost 0.37412 vs LightGBM 0.37464) were treated as near-ties given fold variance. A blend was kept only when it reduced OOF Log Loss.

Validation vs. public vs. private leaderboard.

The OOF score is calculated locally from the training data. The public leaderboard uses only part of the hidden test labels, while the private leaderboard uses the remaining part and determines the final ranking. Therefore, I used cross-validation as the main model-selection criterion rather than relying only on the public leaderboard.

My insight: My validation and public leaderboard rankings agreed — the optimized blend was best on both OOF (0.37330) and Public LB (0.38364) — though public scores ran about 0.01 higher throughout. This mattered because it showed that cross-

validation was reliable, even though the exact scores differed because the public leaderboard used only part of the hidden test set and the blend weights were tuned on OOF predictions. Therefore, I used 5-fold OOF Log Loss as the deciding criterion and the leaderboard only as confirmation when picking the final blend.

5.8 Model Selection

All models were evaluated on the same 5-fold stratified splits, so their OOF Log Loss scores were Forest was not tested because I prioritized stronger boosting models.

Rank	Model	OOF Log Loss	Fold SD	Runtime
1	XGBoost	0.37412	0.00768	~20.8 s
2	LightGBM	0.37464	0.00758	~38.4 s
3	CatBoost	0.38347	0.00669	~8.0 s
4	Logistic Regression	0.45596	0.00975	~0.4 s

Logistic Regression (baseline). (median imputation, missing indicators, scaling, one-hot; C=0.5) — fast but unable to capture non-linear interactions. XGBoost and LightGBM were tried for their non-linear, interaction-aware tree structure (level-wise vs. leaf-wise growth), and CatBoost for its native handling of the 7 nominal categorical features; Random Forest was not tested, as boosting was prioritized. This initial comparison used hand-picked defaults; 5.9 describes a systematic Optuna search that later replaced them. The final submitted model is the Optuna-tuned, OOF-weight-optimised blend of all three boosting models (0.37010), not any single model.

My insight: I observed that All three boosting models comfortably beat Logistic Regression even with default settings, but CatBoost — despite being built for categorical data — scored weakest of the three, both before and after tuning (5.9). This mattered because a model that appears suitable in theory is not always the best in practice. Therefore, I selected models by OOF Log Loss rather than assumption, which is also why CatBoost received only 0.08% weight in the final tuned blend (5.9).

5.9 Hyperparameter Choices

Default hyperparameters (5.8) were replaced with an Optuna search (TPE sampler) run per model, each in two rounds: a Scout round (20 trials, seeds 42/123) checked whether my guessed search ranges were wide enough — a best value landing exactly on a range's edge signals the range is too narrow; a Final round (60 trials, seeds 42/123/2024) then ran on the corrected ranges. The scout round caught real problems: LightGBM's num_leaves hit its lower edge (15) and CatBoost's depth hit its lower edge (4); both ranges were widened downward before the final round. XGBoost's ranges were not changed (no hard edge hit, only colsample_bytree sitting close to its edge in the scout round, which cleared by the final round).

Hyperparameter	XGBoost	LightGBM	CatBoost
Learning rate	0.0238	0.0327	0.0580
Tree size / depth	max_depth = 4	num_leaves = 8	depth = 3
Other	min_child_weight=14, subsample=0.739, colsample=0.516	min_child_samples=30, feature_fraction=0.547, bagging=0.707	l2_leaf_reg=1.81, subsample=0.684
Reg-lambda	reg_lambda = 0.981	reg_lambda = 1.145	
Search range (final round)	unchanged from scout	num_leaves 3–63 (widened from 15–63)	depth 2–8 (widened from 4–8)
Final-round Log Loss	0.3714	0.37089	0.37901

Refit on the shared seed=42 folds (for a fair comparison with 5.8's defaults): XGBoost 0.37412→0.37149, LightGBM 0.37464→0.37094, CatBoost 0.38347→0.37952 — all three improved. Re-blending the tuned models (Nelder–Mead, same method as 5.8) gave weights XGBoost 0.4308 / LightGBM 0.5683 / CatBoost 0.0008 and OOF Log Loss 0.37010, versus 0.37330 for the default-hyperparameter blend.

My insight: I observed that Nearly every boundary check I wrote was an exact-match check (is the value equal to the edge?), which works for integers but is blind to a float value merely sitting close to an edge — I only caught XGBoost's near-edge colsample_bytree by adding a separate tolerance-based check partway through. This mattered because a silent near-edge value could mean the true optimum is still outside the range. I kept the exact-match check for integers (num_leaves, depth) and a tolerance check for floats, but never added a tolerance check for integers too — CatBoost's depth drifted low across both rounds (4 to 3) without ever triggering a warning, which I flag as an open limitation in 5.13 rather than a resolved result (CatBoost's depth kept drifting low even after widening — Round 1 flagged depth=4 correctly, but Round 2's depth=3 (just 1 away from the new edge of 2) triggered no warning, since the check only catches an exact edge hit, not a value merely trending toward it).

5.10 Experimental Results

Exp	EDA insight / hypothesis	Change made	Val. Log Loss (OOF)	Public LB	Decision
0	Logistic Regression baseline	Median imputation + missing indicators + scaling + one-hot; Logistic Regression	0.45596	not submitted	Baseline

Exp	EDA insight / hypothesis	Change made	Val. Log Loss (OOF)	Public LB	Decision
1	XGBoost for nonlinear interactions	Trained XGBoost with early stopping on shared folds, default parameters	0.37412	0.38372	Kept
2	LightGBM: Leaf-wise growth explores a different hypothesis space than level-wise, so errors should be partly complementary	Trained LightGBM with early stopping on shared folds, default parameters	0.37464	0.38480	Kept
3	CatBoost: Data is rich in nominal categoricals; ordered target statistics handle these without manual encoding	Trained CatBoost with early stopping on shared folds, default parameters	0.38347	0.38953	Kept (in blend)
4	Equal blend of XGBoost, LightGBM, and CatBoost. Errors are only partly correlated, so averaging should cancel some model-specific error	Equal-weight blend of XGBoost + LightGBM + CatBoost, default parameters	0.37431	0.38407	Rejected
5	Optimized blend of XGBoost, LightGBM, and CatBoost. Members differ in strength, so unequal weights may beat a plain average	Nelder-Mead weight search on OOF (XGB=0.532, LGBM=0.429, Cat=0.040), default parameters	0.37330	0.38364	Superseded by Experiment 10
6	Defaults are unlikely to be optimal	Optuna, single-pass, 60 trials	not preserved (code later rewritten)	0.38161	Superseded — 2 params hit range edges
7	Exp 6 range may be too narrow — recheck XGBoost	Scout + final round (2-round Optuna)	0.37149	n/a (blend member)	Kept, no edge hit
8	num_leaves range likely too narrow	Scout caught edge (15) → widened → final round	0.37094	n/a (blend member)	Kept, range fix confirmed
9	depth range likely too narrow	Scout caught edge (4) → widened → final round	0.37952	n/a (blend member)	Kept, open caveat (5.13)
10	Unequal weights on tuned models	Nelder-Mead on 3 tuned models	0.37010	0.38042	Selected — final (v3)
11	Fold noise may still affect Exp 10	Re-trained Exp 10 models over 3 seeds, re-blended	0.36713	0.38067	Rejected — CV/LB disagreed

Experiment 1 gave the largest single jump (~18% OOF improvement over the baseline); LightGBM nearly matched XGBoost, while CatBoost was weaker. The equal-weight blend (Exp 4) underperformed XGBoost alone because CatBoost's one-third weight dragged the average down. Optimising the weights (Exp 5) fixed this, and Optuna tuning (Exp 7–10) improved every model further, taking the final blend to 0.37010 / 0.38042 (Exp 10) — the best OOF and Public LB together. Exp 11 (seed-averaging) is the most interesting failure: OOF improved (0.36713) but Public LB got worse (0.38067), a genuine CV/LB disagreement rather than a one-sided failure — both gaps versus Exp 10 are smaller than the fold-to-fold noise (~0.007–0.012) seen throughout the study, so I judged this a tie within noise rather than a real result, and kept the simpler Exp 10 as the final submission.

My insight: I observed that the most useful failed experiment was Exp 11, not because it was wrong, but because CV and the leaderboard pointed in opposite directions — a case my earlier assumption ('trust CV over the leaderboard') did not fully cover. This mattered because blindly trusting CV here would have swapped my final submission for a worse one on the actual scoring set. Therefore, I added a second rule: only trust a CV improvement if it exceeds the fold-noise band, regardless of which direction the leaderboard moved. This did not change my final submission, but it changed how much I trust small CV gains going forward.

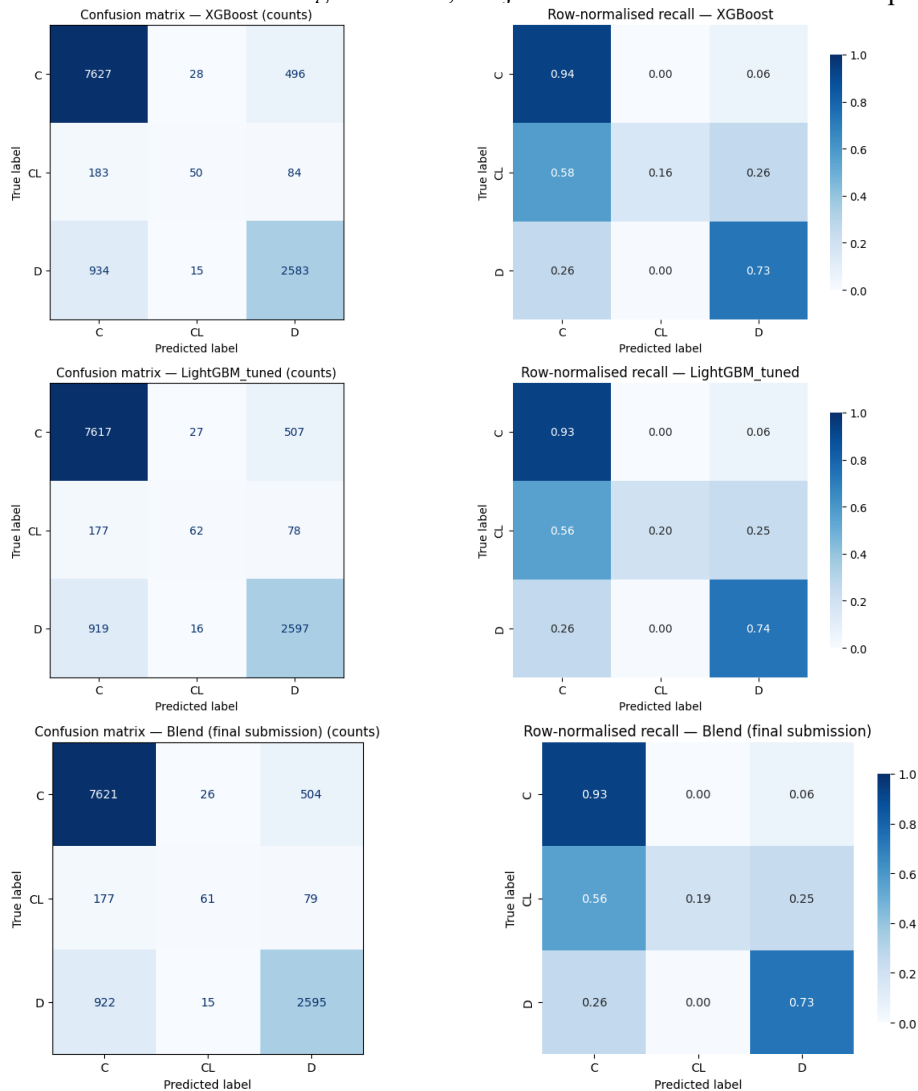
5.11 Final Model Description

The final solution (Steps 12–16) is the OOF-weight-optimised blend of Optuna-tuned XGBoost, LightGBM, and CatBoost (hyperparameters in 5.9; weights XGBoost 0.4308, LightGBM 0.5683, CatBoost 0.0008), trained on all 19 features (12 numeric incl. in_trial, 7 categorical) via the shared 5-fold StratifiedKfold; test predictions are averaged across folds per model, then blended with the same weights. Local OOF Log Loss is 0.37010 (best of all reliable candidates — see 5.13 for the Exp 11 caveat) and Public LB is 0.38042 (also the best reliably-reproducible submission). The file is submission_blend_optimised_v3.csv, generated by make_submission(blend_test_tuned, ...) in Steps 12.7/16 from the blended test predictions computed in Step 12.6; make_submission also validates row count, column layout, probability sums, and NaNs before saving. This model (Exp 10) was chosen over the default-hyperparameter blend (Exp 5, 0.37330) because tuning improved every member model, and over the seed-averaged variant (Exp 11, lower OOF but worse Public LB) because its CV gain did not exceed normal fold noise.

My insight: I observed that every model was scored on the same folds, and the tuned blend's OOF ranking was confirmed by the Public LB (both improved together over the default blend), so I trusted it was genuinely better rather than lucky. The harder trade-off was Exp 11: CV said seed-averaging was better, but Public LB disagreed, and I had to decide which signal to trust with only one submission attempt available to check. I chose the simpler, non-seed-averaged model, reasoning that a real improvement should show up on both signals when the gap is this small. With one more week, I would work on the CL class (5.12), where nearly all remaining error is, rather than continuing to refine the blend.

5.12 Error Analysis

Error analysis (Step 15) compares three stages on the same OOF predictions: "before" tuning (XGBoost, the best default-hyperparameter model, 5.8), "after" tuning (LightGBM_tuned — the best of the 3 individually-tuned models, 0.37094, not XGBoost_tuned as might be expected), and the final blend (Exp 10). This before/after design checks whether tuning fixed the weaknesses found in the original model, not just whether the headline score improved than the final blend.



Class	n	Recall: before / after / blend	Mean LogLoss: before / after / blend
C	8,151	0.936 / 0.934 / 0.935	0.199 / 0.198 / 0.198
CL	317	0.158 / 0.196 / 0.192	2.496 / 2.486 / 2.441
D	3,532	0.731 / 0.735 / 0.735	0.588 / 0.581 / 0.581

Interpretation: C and D barely move — tuning was never going to help there, since they were already well handled. CL recall improved from 15.8% (before) to 19.6% (after) to 19.2% (blend) — a real gain, but CL is still by far the weakest class, with Log Loss over 4× D's even after tuning. Calibration (mean confidence when correct vs. incorrect) is essentially unchanged across all three stages: 0.890/0.718 (before), 0.890/0.717 (after), 0.889/0.716 (blend) — tuning improved discrimination on CL slightly but did not change how well-calibrated the model's confidence is.

My insight: From this step, I observed I expected tuning to meaningfully close the CL gap, since it improved every model's overall Log Loss. Instead, CL recall moved from 15.8% to only ~19%, and the near-identical calibration gap (~0.17 in all three stages) shows tuning did not touch how the model expresses uncertainty — it only sharpened decisions on cases it was already reasonably equipped to handle. This mattered because it confirmed the CL weakness is a data/feature problem (severe class imbalance, overlapping feature distributions with C and D), not a hyperparameter problem — no amount of further tuning within this same feature set is likely to fix it.

5.13 Limitations

Class imbalance — CL is only 2.64% of training data and stays hardest to predict even after tuning (5.12); no class weighting or resampling was tried. Insensitive integer boundary check — the Optuna boundary check (5.9) only flags an exact match to a range's edge for integer parameters; it never caught CatBoost's depth trending low across both rounds (4→3), so that range may still be too narrow. Blend-selection bias — weights were optimised and evaluated on the same OOF predictions, so the reported gain may be mildly optimistic. Seed-averaging left unresolved — Exp 11 (5.10) showed CV and Public LB disagreeing on whether seed-averaging helps; with only one seed pair tested, this could not be settled with confidence. Limited feature engineering — only the missingness-block indicator was tested, with no controlled ablation; Clinical Stage's natural order was not explicitly used. Missing-value uncertainty — the true cause of missingness is unknown; in trial is only an inferred proxy. Limited interpretability — the 3-model blend has no SHAP or feature-importance analysis. Non-Kaggle tuning environment — Optuna search (Step 12) ran on a local MacBook, not the Kaggle CPU used for the rest of the pipeline (5.14), so its ~5.5-hour runtime is not directly comparable.

My insight: From this step, I observed that the biggest weakness is still the unresolved CL class — tuning helped every other metric but barely moved CL recall (5.12), confirming this needs a data/feature-level fix, not more tuning. The decision I'm least confident about is Exp 11: with only one CV/LB data point disagreeing, I cannot rule out that seed-averaging is genuinely better and the Public LB draw was itself noise. Next time, I would test CL-specific features or class weighting first, and run seed-averaging with more seed pairs (not just one comparison) before trusting either signal on a close call like this.

5.14 Reproducibility Information

Python 3.13.5, numpy 2.1.3, pandas 2.2.3, scikit-learn 1.6.1, scipy 1.15.3, xgboost 3.3.0, lightgbm 4.7.0. CatBoost is version 1.2.10.

Random seed: 42 (StratifiedKFold, all model).

Optuna's TPESampler also seeded)

Hardware / platform used: The notebook runs on CPU Macbook Pro; no GPU is required.

local MacBook Pro (macOS, arm64, VS Code + Jupyter) for the Optuna search (Step 12) and seed-averaging (Step 13), since these needed far more wall-clock time than typically available in one Kaggle session — macOS additionally needs Homebrew's libomp installed for XGBoost/LightGBM to import correctly.

Runtime: Optuna tuning ≈ 5.5 hours total across both rounds for all 3 models (measured via Optuna's own trial timestamps); all other model training/refits/blends ≈ 2–3 minutes.

Required files: aio26_train.csv, aio26_test.csv, notebook Kaggle1_Uyen6.ipynb.

Training: run Steps 1–11 (defaults) then Step 12 (Optuna, ~5.5 h — the slow step).

Prediction generation: Step 12.6–12.7 blend the 3 tuned models and build test-set predictions.

Final submission: Step 16 → make_submission() validates and writes submission_blend_optimised_v3.csv (also regenerated by Step 12.7).

Final configuration: Blend = 0.4308·XGBoost + 0.5683·LightGBM + 0.0008·CatBoost (hyperparameters in 5.9); OOF Log Loss = 0.37010, Public LB = 0.38042.

My insight: I observed that Fixing SEED = 42 across StratifiedKFold, all models, and Optuna's sampler made my results reproducible for anyone using the same library versions — refitting the tuned models after fixing a range-swap bug in the CatBoost tuning code (found by re-reading the boundary-check output against the actual code, not just trusting the printed message) reproduced the exact same OOF scores as the original v3 submission, which was strong evidence the bug never actually affected the submitted result. The main reproducibility risk now is the split hardware: Step 12's ~5.5-hour runtime was measured on my MacBook, not Kaggle, so a grader reproducing it on Kaggle CPU should expect a different (likely longer, given Kaggle's shared/free-tier CPU) runtime, even though the final scores should match exactly given the same seeds.

5.15 References and Code Attribution

All notebook code was written originally for this challenge; no external competition solution, public notebook, or repository was copied. References consulted (official docs & course materials): scikit-learn (StratifiedKFold, ColumnTransformer, Pipeline, SimpleImputer, OneHotEncoder, StandardScaler, LogisticRegression, log_loss, confusion_matrix); XGBoost / LightGBM / CatBoost classifier APIs, native categorical handling, early stopping; SciPy (scipy.optimize.minimize, Nelder-Mead) for blend weights; NumPy / pandas / matplotlib / seaborn for data handling and plotting; AIO 2026 course materials (Kaggle Competition 1 lecture, TA session) for the general approach. No external code fragments were reused — every function (build_features, run_cv, the four train_* functions, blend_logloss, make_submission) was written independently, following official API syntax only. Team understanding: individual submission — I understand and can explain every part of the notebook.

5.16 Compliance Declaration

I declare that this submission is my own original work. I did not use GPT, ChatGPT, large language models, generative-AI assistants, AI coding assistants, prohibited external data, pretrained models, paid computing resources, hidden test labels, unauthorized collaboration, or copied competition solutions. All permitted external references and code fragments have been properly cited. I understand that false or misleading information may result in disqualification.

All code, analysis, and this report were produced by me; any external reference used is cited in Section 5.15. This report accurately describes the notebook that generated the final submission file, `submission_blend_optimised_v3.csv`. — Trần Thị Mỹ Uyên, Kaggle username: myuyentran.

6. Required “Insight Trace” Table

Observation	Interpretation	Action	Result	Decision
Target classes imbalanced (CL 2.6%)	Random split risks near-empty CL folds	Shared 5-fold StratifiedKFold	Stable ~2.6% CL per fold	Kept
Missing rate differs by class (up to 11 pts)	Missingness may be informative	Impute + missing indicators (LR)	Signal preserved, not erased	Kept
7 trial columns missing as a block (56.1%/42.9%)	Structural, not random	Engineered <code>in_trial</code> flag	Used by all models	Kept
Numeric features skewed / different scales	Linear model needs scaling; trees don't	StandardScaler for LR only	LR trained properly (0.45596)	Kept
Categorical features nominal	Label encoding creates false order	One-hot (LR); native (trees)	Avoided ordering bias	Kept
Relationships look non-linear / interacting	Linear model can't capture this	XGBoost / LightGBM / CatBoost	OOF: 0.45596 → 0.37412	Kept
3 boosting OOF errors only partly correlated	Blending should cancel some error	Equal-weight blend	Worse (0.37431) than XGB alone	Rejected
CatBoost clearly weaker than the other two	Equal weight drags average down	Nelder–Mead weight search	Best score 0.37330; CatBoost→0.04	Kept
CL confused with C/D (16% recall)	Main remaining error source	Documented for future work	Not yet fixed	Future work
<code>num_leaves</code> /depth hit scout-round range edges	Search ranges guessed too narrow	Widen ranges, run final round (Optuna)	OOF: 0.37330 → 0.37010 blend	Kept
Seed-averaged blend: OOF better, Public LB worse	CV/LB disagreement; gap < fold-noise band	Compared gap size to known fold std (~0.007–0.012)	Kept simpler v3 (0.38042) over 0.38067	Rejected

7. Required “What I Learned” Section

Before this challenge, I thought picking the right model mattered most. The results proved me wrong: moving from Logistic Regression to gradient boosting cut OOF Log Loss from 0.45596 to 0.37412 (~18%), but the three boosting models were almost tied, and later, systematic hyperparameter tuning (Optuna) closed nearly as much of the remaining gap (0.37330→0.37010) as switching models did in the first place. My biggest lesson was that EDA, preprocessing, honest validation, and tuning all matter roughly as much as the model choice itself.

The most useful failure was not blending (equal weights briefly underperformed XGBoost alone before I optimised the weights), but seed-averaging: it improved OOF (0.37010→0.36713) but made the Public LB worse (0.38042→0.38067) — my first real case of CV and the leaderboard flatly disagreeing. I had assumed 'trust CV over the leaderboard' was a safe rule; this taught me it needs a caveat — only trust a CV gain that's larger than the fold-to-fold noise already observed, regardless of which way the leaderboard moves.

I also learned that my own tooling can hide bugs from me: my Optuna boundary check caught integer parameters landing exactly on a range edge, but was blind to a float sitting close to one, and later I found a range-swap bug in the CatBoost tuning code that only surfaced when I compared the printed warning against the actual code. Refitting after the fix reproduced the exact same score as before, which taught me to verify refactored code against a known-good baseline rather than trusting that a rewrite preserved behavior. With one more week, I would tackle the CL class directly, since tuning barely moved its recall (5.12) despite improving every other metric.